

PLACEIQ

AI-DRIVEN CAMPUS PLACEMENT MANAGEMENT AND PREPARATION SYSTEM

A project report submitted in partial fulfillment of the requirement for
the award of the Degree of
BACHELOR OF SCIENCE IN COMPUTER SCIENCE



Submitted by

MALA M : 20524UCSC075

: :
: :

Under the Guidance of

Dr. S. MANIMEKALAI, M.Sc, M. Phil., Ph.D

Dean & Associate Professor

PG and Research Department of Computer Science
THEIVANAI AMMAL COLLEGE FOR WOMEN
(AUTONOMOUS)

(Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited by NAAC (4th Cycle) at 'A+' Grade)

(Recognized Under 2(f) and 12(B) by UGC)

Villupuram - 605 403

NOV - 2026

THEIVANAI AMMAL COLLEGE FOR WOMEN
(AUTONOMOUS)

(Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited by NAAC (4th Cycle) at 'A+' Grade)

(Recognized Under 2(f) and 12(B) by UGC)

Villupuram - 605 403



BONAFIDE CERTIFICATE

This is to certify that the project entitled "PLACEIQ - AI-DRIVEN CAMPUS PLACEMENT MANAGEMENT AND PREPARATION SYSTEM" is a bonafide work carried out by MALA M (REGISTER NO: 20524UCSC075) in partial fulfillment of the requirement for the award of the Degree of Bachelor of Science in Computer Science, during the academic year 2026 - 2027.

Faculty Supervisor

Head of the Department

Dean of Computer Science

Submitted for Viva-voce examination held on _____

Internal Examiner

External Examiner

DECLARATION

I hereby declare that the project entitled "PLACEIQ - AI-DRIVEN CAMPUS PLACEMENT MANAGEMENT AND PREPARATION SYSTEM" was carried out by me from June 2026 to November 2026 under the guidance of Dr. S. MANIMEKALAI, M.Sc, M.Phil., Ph.D, Dean & Associate Professor, PG and Research Department of Computer Science, Theivanai Ammal College for Women (Autonomous), Villupuram.

I also declare that this project report is my original work and has not been submitted, either wholly or in part, for the award of any degree, diploma or other similar title or recognition in this or any other University/Institution.

Place: Villupuram

Date:

MALA M
(20524UCSC075)

ACKNOWLEDGEMENT

First and foremost, I would like to express my deepest gratitude to the Almighty for blessing me with the wisdom and perseverance to complete this project successfully.

I extend my sincere thanks to Thiru. E. Swamikkannu, Founder-Chairman of the E.S Group of Institutions, and the management for providing the infrastructure and encouragement needed to carry out this work.

I am deeply grateful to Dr. J. Kalaimathi, Principal, and Ms. V. S. Selvi, Vice Principal, for their continuous motivation and support throughout the course of this project.

I would also like to express my heartfelt thanks to Dr. S. Manimekalai, Dean & Associate Professor of the PG and Research Department of Computer Science, for her valuable suggestions and encouragement.

My sincere appreciation goes to Ms. K. Manohari, Head and Assistant Professor of the PG and Research Department of Computer Science, for her constant support and timely help.

I am especially grateful to my Project Guide, Dr. S. Manimekalai, for her effective guidance, insightful feedback and patient supervision at every stage of this project.

Finally, I would like to thank my parents, friends and well-wishers, whose unwavering support has been instrumental in the successful completion of this project.

MALA M

(20524UCSC075)

CONTENT

Chapter No	PARTICULARS	PAGE. NO
	Abstract	
I	Introduction Background Problem Statement Objectives Scope of the Project Significance of the Study	
II	Literature Review	
III	System Analysis Existing System Proposed System Feasibility Study System Goals and Objectives	
IV	System Requirements Hardware Requirements Software Requirements	
V	Technology Description Angular and TypeScript Ionic Framework Capacitor Android Platform	
VI	System Modules Module Description Algorithms Module Integration	
VII	System Design System Architecture Application Flow Data Design	
VIII	System Implementation Coding Screenshots	
IX	System Testing Unit Testing Integration Testing System Testing User Acceptance Testing Performance Testing	
X	Conclusion Summary of the Work Project Impact Future Enhancements	
	References	

PLACEIQ

AI-DRIVEN CAMPUS PLACEMENT MANAGEMENT AND PREPARATION SYSTEM

ABSTRACT

In most colleges, information about placement drives is passed to students through notice boards, class representatives and messaging groups. Because of this, students often come to know about a drive late, apply for companies whose conditions they do not satisfy, or miss the last date completely. Preparing for aptitude, coding and interview rounds also happens without any structured support from the institution. The PlaceIQ application was developed to bring these activities into a single mobile application.

The application is built using Angular with the Ionic user-interface library and is packaged as an Android application using Capacitor. Students and the placement officer see different screens based on their role. Students can view open drives, see whether they satisfy each company's conditions, submit applications, follow preparation courses, check their readiness score and ask the in-app assistant for help. The officer can add or edit company drives, view student records and publish announcements. Eligibility is checked inside the app by comparing CGPA, department, arrears and skills with the company's conditions. A weighted scoring method produces a readiness score along with suggestions, and a keyword-based chatbot answers common placement doubts and can also report live values such as eligible drives and deadlines.

The application gives each student a clear view of open drives, eligibility status, preparation progress and readiness without waiting for manual verification. It also reduces the repeated checking work of the placement cell. Since all data is stored on the device, the application works fully offline and can later be connected to a central server and a trained prediction model without changing the screens.

CHAPTER – 1

INTRODUCTION

1.1 BACKGROUND

During every placement season, the placement cell of a college shares drive details with students, collects applications and verifies whether each applicant satisfies the company's conditions. Students, on their side, have to watch several channels for announcements, work out eligibility on their own and prepare for tests and interviews without a common platform. In most institutions this is still done through notice boards, circulars, spreadsheets and messaging groups, so information is frequently delayed or missed.

A mobile application is a practical way to reach students because almost every student carries a smartphone. Frameworks such as Ionic and Capacitor allow one web codebase to be packed as a normal Android application, so development cost stays low. In the same way, simple rule-based and scoring methods can provide useful guidance — eligibility checking, readiness scoring and question answering — directly on the device, without needing a server.

PlaceIQ uses this approach for the placement problem. It keeps all drive information in one place, checks eligibility automatically, gives each student a readiness score based on weighted factors, provides a course list with progress tracking for preparation, and includes a chatbot that answers common doubts at any time.

1.2 PROBLEM STATEMENT

The existing placement workflow suffers from three concrete problems. First, information is scattered: drive announcements, eligibility criteria and deadlines reach students through disconnected channels, so deadlines are missed and ineligible students apply while eligible students remain unaware. Second, eligibility verification is manual: the placement officer must compare each applicant's CGPA, department, arrear history and skills against every company's criteria. Third, preparation support is unstructured: students preparing for aptitude tests, coding rounds and interviews have

no guided material inside the system and nowhere to get instant answers to common doubts.

There is a need for a single mobile platform that gives each student a personalised, real-time view of placement drives, automatically evaluates eligibility, quantifies readiness, guides preparation through curated course content, and answers frequent questions instantly — while giving the placement cell a simple console to manage drives, students and announcements.

1.3 OBJECTIVES

The primary objectives of this project are:

- To design and develop a mobile application that centralises all campus placement drives with role, package, eligibility criteria and deadlines.
- To automate eligibility verification by evaluating each student's CGPA, department, arrear count and skills against company criteria in real time.
- To compute an AI readiness score for every student using a weighted factor model covering academics, skills, arrears, certifications and department demand, with actionable suggestions.
- To provide a preparation module offering curated courses (aptitude, DSA, SQL, soft skills, interview skills, company-wise preparation) with enrolment and lesson-level progress tracking.
- To build an offline AI-style chatbot that answers predefined placement questions through keyword matching and can also report live data such as eligible drives and deadlines.
- To equip the placement cell with an admin console for managing companies, student records and notification broadcasts.

1.4 SCOPE OF THE PROJECT

The scope of this project includes:

- Student module — personalised dashboard, drive listing with eligibility status, one-tap application, AI readiness score, preparation courses, doubt-clarification chatbot and alerts.

- Admin module — placement-officer dashboard with applicant analytics, company drive CRUD, student record browsing and notification publishing.
- Android deployment — the application is packaged as an installable APK using Capacitor and the Android SDK.
- Offline data layer — all entities are persisted in the device's localStorage through a seeded JSON store, so the application runs fully without a network connection.

The current build uses demonstration data and a rule-based engine in place of a server-side machine-learning model; it is designed so that a REST backend and a trained model can replace these layers without changing the user interface.

1.5 SIGNIFICANCE OF THE STUDY

The project contributes a practical, deployable solution to a real institutional problem. For students, it removes the uncertainty around eligibility and deadlines and converts placement preparation into a guided, trackable activity. For the placement cell, it eliminates repetitive manual verification and provides visibility into applicant counts and student readiness. The architecture — web technologies delivered as a native app with an on-device intelligence layer — demonstrates a low-cost pattern that small institutions can adopt without server infrastructure.

CHAPTER – 2

LITERATURE REVIEW

The following works were surveyed to understand existing approaches to placement prediction, campus recruitment management and educational guidance systems.

Literature Survey 1

Title : Student Placement Prediction using Machine Learning
Authors : S. Harinath, A. Prasad, Suma H. S., Suraksha A., Tojo Mathew
Published Year : 2019

Description

Published in IRJET (Vol. 6, Issue 4), this work applies Naive Bayes and K-Nearest Neighbour classifiers to historical placement data to predict whether a student will be placed, and also predicts the probable company. It demonstrates how the placement cell can identify students needing attention early.

Drawbacks

Requires a labelled historical dataset and offline model training; it predicts an outcome but gives the student no actionable guidance on how to improve it.

Literature Survey 2

Title : Automatic Student Analysis and Placement Prediction using Advanced Machine Learning Algorithms
Authors : K. Anvesh, B. S. Prasad, V. V. Sai, R. Laxman, B. S. Narayana
Published Year : 2019

Description

Published in IJITEE, this paper evaluates several machine learning algorithms for automated analysis of student academic records and placement outcome prediction, showing that ensemble-style classifiers outperform single models on campus datasets.

Drawbacks

Prediction is batch-oriented and opaque — the factors driving each prediction are not explained to the student, limiting its usefulness as a self-improvement tool.

Literature Survey 3

Title : A Comparative Study of Artificial Intelligence Models for Predicting Campus Placement Outcomes in Higher Education Institutions

Authors : IJERT Research Paper

Published Year : 2025

Description

Compares Logistic Regression, Decision Tree, SVM, Random Forest and Gradient Boosting on features such as CGPA, internship exposure, projects, technical proficiency, communication ability and backlogs; ensemble models consistently outperform single classifiers.

Drawbacks

Evaluates models in isolation with no deployable application — no student-facing interface, drive workflow or real-time eligibility checking is provided.

Literature Survey 4

Title : Enhanced Student Placement Prediction Using Machine Learning: A Comparative Evaluation of Algorithms

Authors : IJETT Research Paper

Published Year : 2025

Description

Benchmarks Logistic Regression, Random Forest, Decision Tree, Naive Bayes, SVM, KNN, Gradient Boosting and LDA on the College Placement Predictor dataset, reporting around 94% accuracy for KNN, Logistic Regression and SVM.

Drawbacks

Accuracy-oriented benchmarking only; the approach needs a hosted dataset and compute, and offers no per-student improvement pathway or placement-process integration.

Literature Survey 5

Title : Predicting College Students' Placements Based on Academic Performance Using Machine Learning Approaches

Authors : IJMECS (MECS Press)

Published Year : 2023

Description

Published in IJMECS Vol. 15 No. 6, this study trains Logistic Regression, Gaussian Naive Bayes, Random Forest, SVM and KNN on academic markers, identifying Random Forest as the most effective for forecasting placements from academic performance.

Drawbacks

Relies almost entirely on academic markers — recruiter criteria, skills and certifications are outside the model, and no application workflow accompanies the prediction.

Literature Survey 6

Title : A Chatbot Student Support System in Open and Distance Learning

Institutions

Authors : J. N. Ndunagu, C. U. Ezeanya, B. O. Onuorah, J. C. Onyeakazi, E. Ukwandu

Published Year : 2025

Description

Published in Computers (MDPI), 14(3):96, this work develops a ChatterBot-based assistant for the National Open University of Nigeria; in a survey of 579 students, 64% of respondents rated the chatbot extremely helpful in resolving queries and complaints.

Drawbacks

A general-purpose campus FAQ bot with no access to student-specific placement data such as eligibility, deadlines or applications; it also depends on a server-hosted framework.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing placement workflow, drive announcements are circulated through notice boards, class representatives and messaging groups. Students manually check whether they satisfy each company's criteria — CGPA cutoffs, eligible departments, arrear limits and required skills — and submit their names through forms or spreadsheets. The placement officer then verifies every applicant record by hand before forwarding lists to the recruiter.

DISADVANTAGES OF THE EXISTING SYSTEM

- Announcements are easily missed; there is no single source of truth for open drives.
- Eligibility checking is manual and error-prone for both students and the placement cell.
- Students cannot see why they are ineligible or what to improve.
- No integrated preparation support — students depend on scattered external material.
- Frequent doubts (patterns, rules, deadlines) consume placement-officer time.
- No consolidated view of applications, notifications and student readiness.

3.2 PROPOSED SYSTEM

The proposed system is a hybrid mobile application that puts the complete placement workflow in the student's pocket. Drives are published by the placement cell through an admin console and appear instantly in the student app with automatic eligibility evaluation. Each student receives an explainable AI readiness score with improvement suggestions. A curated course catalogue with enrolment and lesson-level progress tracking guides preparation, and an offline chatbot answers common doubts and reports live status such as eligible drives, deadlines and applications.

ADVANTAGES OF THE PROPOSED SYSTEM

- Single mobile source of truth for every drive, criterion and deadline.
- Instant, transparent eligibility evaluation with per-criterion pass/fail detail.
- Explainable readiness score plus concrete improvement suggestions.
- Structured preparation content with measurable progress.
- 24×7 doubt clarification through the chatbot without staff involvement.
- Admin console reduces manual work for drive and notification management.
- Fully offline operation — no server or internet dependency for the demo deployment.

3.3 FEASIBILITY STUDY

A feasibility study evaluates the practicality of the proposed system across three dimensions.

Technical Feasibility

The system is built entirely with mature, freely available technologies — Angular, Ionic, TypeScript and Capacitor — and packaged with the Android SDK. All required components (rule engine, scoring model, keyword chatbot) run on-device, so no additional infrastructure is required. The technology stack is technically feasible.

Economical Feasibility

Development uses open-source frameworks with zero licensing cost. Deployment requires only a standard Android device; localStorage persistence removes database server costs. The total cost of the project is limited to development effort, making it highly economical.

Operational Feasibility

The interface follows familiar mobile-app patterns — tab navigation, cards, and chat — requiring no training for students. The placement officer's console mirrors existing spreadsheet workflows, so adoption is straightforward. The system is operationally feasible.

3.4 SYSTEM GOALS AND OBJECTIVES

- Reduce the time between drive announcement and student application to near zero.
- Ensure every student can see exactly which criteria they pass or fail for each drive.
- Provide a measurable, explainable readiness indicator to focus preparation effort.
- Keep all placement interactions — learning, applying, asking — inside one application.
- Give the placement cell a lightweight console with applicant and broadcast management.

CHAPTER – 4

SYSTEM REQUIREMENTS

A system requirement defines the hardware and software environment needed to develop and run the proposed system.

4.1 HARDWARE REQUIREMENTS

Development machine:

- Processor : Intel Core i3 / equivalent or higher
- RAM : 8 GB minimum (16 GB recommended)
- Hard disk : 256 GB SSD with ~10 GB free for SDK and build tools
- Display : 1366 × 768 or higher

Target device (for running the APK):

- Android smartphone or emulator
- Android 8.0 (API level 26) or above
- 2 GB RAM, ~20 MB free storage

4.2 SOFTWARE REQUIREMENTS

- Operating System : Windows 10/11 (64-bit)
- Runtime : Node.js v22, npm v10
- Framework : Angular 20, Ionic 9 (@ionic/angular)
- Native Bridge : Capacitor 8.5 (@capacitor/core, @capacitor/android)
- Build Tools : Android SDK (API 36), Gradle 8.14 (wrapper)
- JDK : OpenJDK 21
- IDE : Visual Studio Code / Android Studio
- Language : TypeScript 5.9, HTML5, SCSS
- Data Storage : Browser/WebView localStorage (JSON store)

CHAPTER – 5

TECHNOLOGY DESCRIPTION

5.1 ANGULAR AND TYPESCRIPT

Angular is used in this project to build all the screens and their navigation. Each page of the app is written as a standalone component with its own template and style file, and the Angular Router decides which page is shown for each address. Services such as DataService, PredictionService and ChatbotService hold the shared logic and are supplied to pages through dependency injection. TypeScript is used for all program files so that entities like Student, Company, Course and Enrollment have a fixed structure throughout the app. A page component in this project looks like:

```
@Component({
  selector: 'app-courses',
  templateUrl: 'courses.page.html',
  styleUrls: ['courses.page.scss'],
  imports: [IonicModule, CommonModule, FormsModule],
})
export class CoursesPage { ... }
```

5.2 IONIC FRAMEWORK

Ionic supplies the ready-made mobile components used on every screen — tab bars, cards, search fields, modals, floating buttons, accordion lists and progress bars. These components already behave like native Android controls, so the application gets a normal mobile look without writing separate native screens. The company detail and course detail views in this project, for example, are Ionic sheet modals opened from the list pages.

5.3 CAPACITOR

Capacitor is used to turn the finished web build into an installable Android application. It copies the compiled www folder into a generated Android project and runs the app inside the system WebView, while still allowing access to device features when needed. The configuration used in this project is:

```
const config: CapacitorConfig = {
  appId: 'com.placementapp.dev',
```

```
    appName: 'placement-app',  
    webDir: 'www'  
};
```

5.4 ANDROID PLATFORM AND GRADLE

The Android project created by Capacitor is compiled with the Gradle build tool using the Android Gradle Plugin against SDK API 36. Running the `assembleDebug` task produces `app-debug.apk`, which can be installed directly on a phone for testing. The `WebView` inside the app hosts the Ionic screens at runtime, and the `WebView`'s `localStorage` is used as the on-device store for students, companies, applications, notifications and enrolments.

CHAPTER – 6

SYSTEM MODULES

The system is divided into functional modules to simplify development, testing and maintenance. Each module handles a distinct responsibility.

MODULE 1: AUTHENTICATION MODULE

- Role-based login (student / placement officer) with credential validation.
- Session persistence in localStorage; route guards restrict student and admin areas.

MODULE 2: STUDENT DASHBOARD MODULE

- Personalised greeting, CGPA/department snapshot and application statistics.
- Recommended eligible drives, upcoming deadlines and one-tap navigation.

MODULE 3: PLACEMENT DRIVES MODULE

- Searchable, filterable drive list with company, role, package and deadline.
- Detail view showing the four-criterion eligibility check and missing-skill chips.
- One-tap application recording with duplicate-application protection.

MODULE 4: AI READINESS PREDICTION MODULE

- Weighted scoring model across six factors — CGPA (40), skill profile (20), arrear history (15), certifications (10), department demand (10) and placement history (5).
- Produces a 0–100 score with High/Moderate/Low label and targeted suggestions.

MODULE 5: COURSES AND ENROLLMENT MODULE

- Catalogue of seven curated preparation courses grouped by category and level.
- Course detail shows outcomes, module/lesson syllabus and duration.
- Enrolment persists per student; lessons can be marked complete with progress bars.

MODULE 6: AI CHATBOT MODULE

- Keyword-matching engine over ~20 predefined question-answer entries covering aptitude, DSA, SQL, resume, HR/GD rounds and company-specific preparation.
- Dynamic intents answer with live data — eligible drives, deadlines, applications, unread alerts, readiness score and profile summary.
- Simulated typing indicator, timestamps and contextual suggestion chips reproduce a real chat experience.

MODULE 7: NOTIFICATION MODULE

- Drive, deadline and announcement alerts with unread badge on the tab bar.
- Mark-all-read support; admin-published notifications appear instantly.

MODULE 8: ADMIN MODULE

- Dashboard with applicant and placement analytics.
- Company CRUD with eligibility criteria, student record browser, notification publishing.

MODULE 9: DATA PERSISTENCE MODULE

- Single-versioned JSON store (students, companies, notifications, applications, enrolments) persisted to localStorage on every mutation.
- Seed routine provides realistic demo data on first launch.

ALGORITHM — ELIGIBILITY CHECK

```

Step 1: Read student record (CGPA, department, arrears, skills).
Step 2: Read company criteria (minCgpa, departments, maxArrears,
requiredSkills).
Step 3: Evaluate CGPA, department, arrears and skill-set match.
Step 4: Compute skillMatchPct and collect missing skills.
Step 5: Mark drive ELIGIBLE only if all four checks pass.
Step 6: Render per-criterion pass/fail detail to the student.
```

ALGORITHM — READINESS SCORE

```

Step 1: cgpaScore = (CGPA / 10) x 40.
Step 2: skillScore = min(20, skills/6 x 12 + inDemandSkills x 2).
Step 3: arrearScore = 15 if no arrears, 7 if <=2, else 2.
Step 4: certScore = min(10, certifications x 4).
Step 5: deptScore = demand rating of department (6-10).
Step 6: histScore = 5 if placed else trend-based 1-3.
```

Step 7: score = sum of factors; label High ≥ 75 , Moderate ≥ 50 .
Step 8: Emit improvement suggestions for weak factors.

ALGORITHM — CHATBOT ANSWER SELECTION

Step 1: Normalise the question (lowercase, tokenise).
Step 2: Score every QA entry = sum of matched keyword weights.
Step 3: Select the entry with the highest positive score.
Step 4: If a dynamic intent, build the answer from live store data.
Step 5: Otherwise return the predefined answer text.
Step 6: If no entry matched, return the fallback guidance message.
Step 7: Attach contextual follow-up suggestions.

MODULE INTEGRATION

All modules communicate through the shared DataService, PredictionService, AuthService and ChatbotService injected via Angular dependency injection. Pages never touch storage directly; this loose coupling allows the data layer to be replaced by a REST API later without modifying the UI modules.

CHAPTER – 7

SYSTEM DESIGN

7.1 SYSTEM ARCHITECTURE

PlaceIQ follows a layered architecture. The presentation layer contains the Ionic/Angular pages for students and the placement officer. The application layer holds the Angular services — authentication, data access, prediction and the chatbot engine — plus the router guards. The native bridge layer packages the web build inside a Capacitor Android WebView and produces the installable APK through Gradle. The data layer persists the JSON store in localStorage and bundles the static course and chatbot knowledge bases.

PlaceIQ — System Architecture

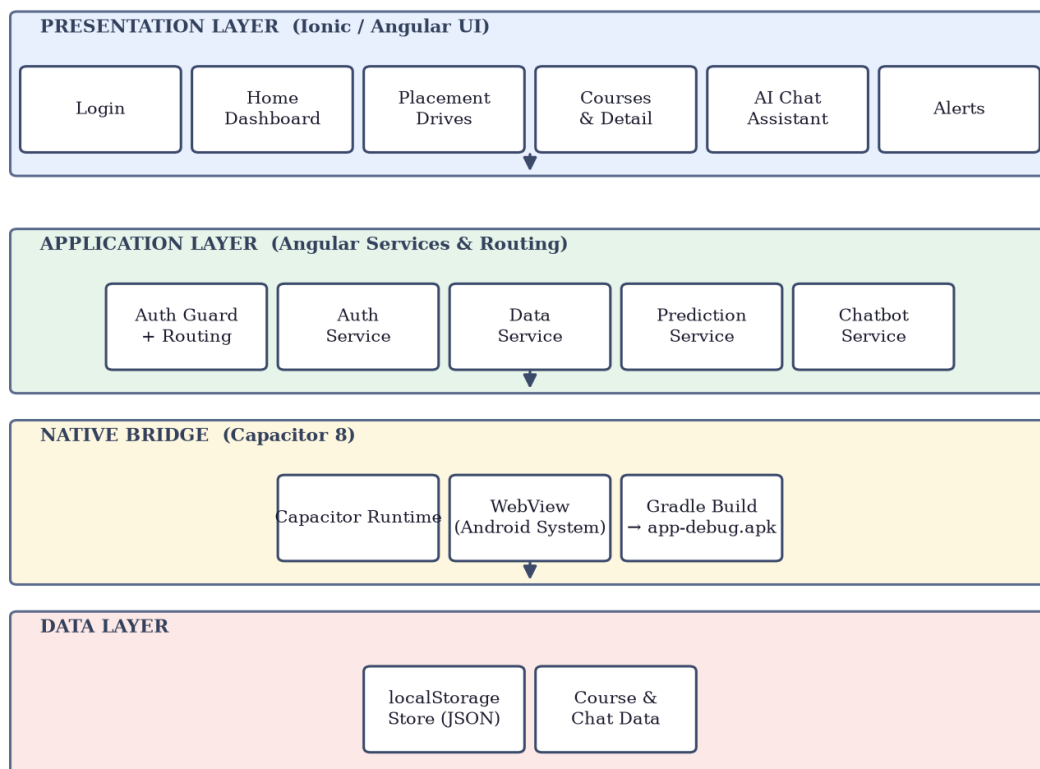


Fig 7.1 — Layered architecture of the PlaceIQ application

7.2 APPLICATION FLOW

On launch the user authenticates; a role guard routes students to the student tab navigator and officers to the admin console. Every student screen reads and writes through the shared DataService, which serialises the entire store to localStorage after each change.

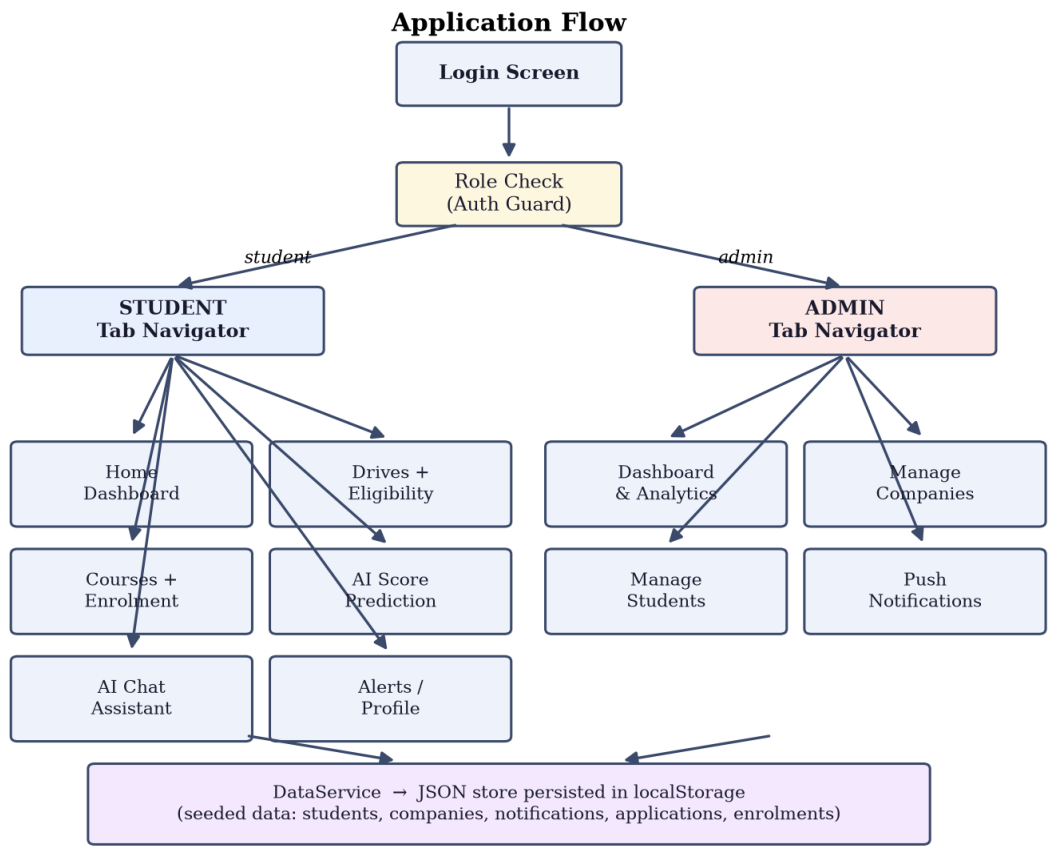


Fig 7.2 — Application flow after role-based login

7.3 DATA DESIGN

The localStorage store contains five entities. STUDENT and COMPANY are related through the APPLICATION join entity (a student applies to many companies). STUDENT relates to COURSE through ENROLLMENT, which also stores the list of completed lesson ids for progress tracking. NOTIFICATION is broadcast to all students, and the CHAT QA knowledge base is a static dataset consulted by the chatbot engine.

Data Model (localStorage store)

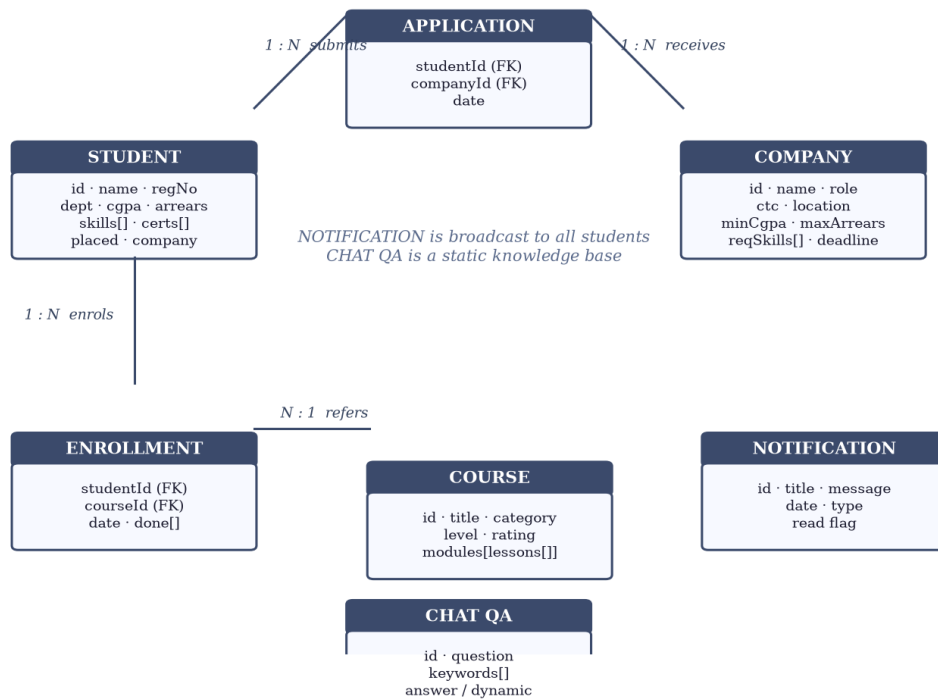


Fig 7.3 — Entity relationship view of the localStorage store

ENTITY	ATTRIBUTES
STUDENT	id (PK), name, email, registerNo, department, cgpa, arrears, skills[], certifications[], phone, placed, placedCompany, placedRole, placedPackage, color
COMPANY	id (PK), name, role, ctc, location, domain, minCgpa, departments[], requiredSkills[], maxArrears, deadline, applyLink, description, openings, color
APPLICATION	studentId (FK), companyId (FK), date
ENROLLMENT	studentId (FK), courseId (FK), date, done[] (completed lesson ids)
NOTIFICATION	id (PK), title, message, date, type (drive/deadline/announcement), read
COURSE	id (PK), title, category, level, description, outcomes[], icon, color, rating, learners, modules[lessons[id, title, minutes, points[]]]
CHAT_QA	id (PK), question, keywords[], answer / dynamicIntent, suggestions[]

Table 7.1 — Data dictionary of the persistent store

CHAPTER – 8

SYSTEM IMPLEMENTATION

8.1 IMPLEMENTATION ENVIRONMENT

The application was developed in Visual Studio Code using Angular 20 standalone components and the Ionic lazy-loaded module. The production web bundle is generated by "ng build" into the www directory, copied into the native project by "npx cap sync android", and compiled to app-debug.apk by the Gradle wrapper inside the generated android project.

8.2 CODING

Key implementation excerpts are reproduced below.

Eligibility evaluation (PredictionService):

```
checkEligibility(s: Student, c: Company): EligibilityResult {
  const missingSkills = c.requiredSkills.filter(k =>
!s.skills.includes(k));
  const checks = [
    { label: 'CGPA Criteria',    passed: s.cgpa >= c.minCgpa },
    { label: 'Department',      passed:
c.departments.includes(s.department) },
    { label: 'Arrears',         passed: s.arrears <= c.maxArrears },
    { label: 'Required Skills', passed: missingSkills.length === 0 },
  ];
  return { eligible: checks.every(k => k.passed), checks, ... };
}
```

Chatbot answer selection (ChatbotService):

```
reply(input: string): BotReply {
  const tokens = new Set(input.toLowerCase().split(/\s+/));
  let best = null, bestScore = 0;
  for (const e of QA_ENTRIES) {
    const score = e.keywords.reduce((s, k) =>
      s + (tokens.has(k) || input.includes(k) ? k.split(' ').length : 0),
    0);
    if (score > bestScore) { bestScore = score; best = e; }
  }
  return best ? build(best) : { text: FALLBACK_ANSWER };
}
```

Enrolment and progress tracking (DataService):

```
enroll(studentId, courseId) {  
  if (!this.enrollmentOf(studentId, courseId)) {  
    this.store.enrollments.push({ studentId, courseId,  
      date: new Date().toISOString(), done: [] });  
    this.persist();  
  }  
}
```

8.3 SCREENSHOTS

Application screens captured from the Android build:

Fig 8.1 — Login screen with role selection

[screenshot]

Fig 8.2 — Student home dashboard with AI score ring and recommendations

[screenshot]

Fig 8.3 — Placement drives list with eligibility badges and skill match

[screenshot]

Fig 8.4 — Drive detail modal with per-criterion eligibility check

[screenshot]

Fig 8.5 — Courses catalogue with categories and progress

[screenshot]

Fig 8.6 — Course detail with module/lesson syllabus and enrolment

[screenshot]

Fig 8.7 — AI chat assistant with suggestion chips and typing indicator

[screenshot]

Fig 8.8 — AI readiness score factor breakdown and suggestions

[screenshot]

Fig 8.9 — Admin dashboard and company management console

[screenshot]

CHAPTER – 9

SYSTEM TESTING

Testing was carried out at five levels to verify that the application behaves correctly under both normal and boundary conditions.

9.1 UNIT TESTING

Individual services were exercised in isolation. The eligibility engine was tested against students meeting and failing each criterion; the prediction model was verified to produce scores in the 0–100 range with the correct High/Moderate/Low labels; the chatbot matcher was verified to select the correct QA entry for single-word and phrased questions and to return the fallback message for out-of-domain input.

9.2 INTEGRATION TESTING

Module interactions were validated end-to-end: applying to a drive updates the applications list and disables the apply button; enrolling in a course activates lesson check-off and the progress bar; publishing a notification in the admin console raises the unread badge on the student tab; chatbot dynamic intents reflect the current store contents.

9.3 SYSTEM TESTING

The complete APK was installed on Android devices and exercised as a whole — login, tab navigation, modal flows, chat session, logout and data persistence across restarts (closing and reopening the app preserves applications, enrolments and read states).

9.4 USER ACCEPTANCE TESTING

Sample users were asked to complete typical tasks — find an eligible drive, apply, enrol in a course and ask the chatbot a question. All tasks were completed without assistance, and feedback on label wording and progress visibility was incorporated.

9.5 PERFORMANCE TESTING

The production bundle loads in under two seconds on a mid-range device; drive lists, course catalogue and chat responses render without perceptible lag. The application occupies ~5 MB installed and uses no network access at runtime.

9.6 TEST CASES

ID	Test Case	Steps / Input	Expected Result	Status
TC01	Valid student login	Enter student credentials and submit	Student dashboard opens	Pass
TC02	Invalid login	Enter wrong password	Error toast shown, stays on login	Pass
TC03	Eligibility — pass	Open drive where all criteria met	Eligible badge; Apply enabled	Pass
TC04	Eligibility — fail	Open drive with CGPA below cutoff	Failed check shown; Apply disabled	Pass
TC05	Apply to drive	Tap Apply Now on an eligible drive	Application recorded; button disabled	Pass
TC06	Course enrolment	Tap Enroll on a course	Enrolled badge; lessons become checkable	Pass
TC07	Lesson progress	Mark a lesson complete	Progress % increases; persists on reopen	Pass
TC08	Chatbot static QA	Ask "Tips for my resume"	Relevant predefined answer + chips	Pass
TC09	Chatbot dynamic QA	Ask "Which companies am I eligible for?"	Live list from eligibility engine	Pass
TC10	Chatbot fallback	Ask an unrelated question	Fallback guidance with suggestions	Pass
TC11	Admin drive CRUD	Add/edit/delete a company drive	Drive list updates for students	Pass
TC12	Notification broadcast	Publish alert from admin console	Unread badge increments on student tab	Pass

Table 9.1 — Summary of executed test cases

CHAPTER – 10

CONCLUSION

SUMMARY OF THE WORK

The PlaceIQ project set out to replace a fragmented, manual campus placement workflow with a single mobile application, and that objective was achieved. The delivered system centralises drive announcements, evaluates every student against every company's eligibility criteria in real time, quantifies placement readiness through an explainable weighted model, guides preparation with a structured course catalogue, answers doubts instantly through an offline chatbot, and gives the placement cell a dedicated admin console. The entire application was built with Angular and Ionic and shipped as a native Android APK through Capacitor.

PROJECT IMPACT

For students, the application turns an opaque process into a transparent, self-service one — eligibility is visible at a glance, deadlines cannot be missed, preparation is measurable, and common doubts are resolved in seconds. For the placement cell, repetitive eligibility screening and announcement circulation are eliminated, freeing officer time for recruiter engagement. Because the system runs fully offline, it can be deployed in institutions with limited infrastructure at effectively zero cost.

FUTURE ENHANCEMENTS

- Cloud backend — replace the localStorage store with a REST API and a central database so drives, applications and enrolments synchronise across devices.
- Trained prediction model — substitute the weighted scoring rules with a machine-learning model trained on historical placement outcomes.
- Push notifications — use Firebase Cloud Messaging for real-time drive and deadline alerts.
- Smarter chatbot — connect the chatbot to a server-side language model for open-domain question answering while retaining the offline fallback.

- Resume analysis — parse uploaded resumes to auto-fill skills and suggest improvements.
- iOS build — Capacitor allows the same codebase to be packaged for Apple devices.
- Analytics dashboard — readiness trends and placement statistics for the institution.

REFERENCES

- [1] S. Harinath, A. Prasad, Suma H. S., Suraksha A. and T. Mathew, "Student Placement Prediction using Machine Learning", International Research Journal of Engineering and Technology (IRJET), Vol. 6, Issue 4, pp. 4577-4579, April 2019.
- [2] K. Anvesh, B. S. Prasad, V. V. Sai, R. Laxman and B. S. Narayana, "Automatic Student Analysis and Placement Prediction using Advanced Machine Learning Algorithms", International Journal of Innovative Technology and Exploring Engineering (IJITEE), 2019.
- [3] "A Comparative Study of Artificial Intelligence Models for Predicting Campus Placement Outcomes in Higher Education Institutions", International Journal of Engineering Research & Technology (IJERT), 2025.
- [4] "Enhanced Student Placement Prediction Using Machine Learning: A Comparative Evaluation of Algorithms", International Journal of Engineering Trends and Technology (IJETT), Vol. 73, Issue 1, 2025.
- [5] "Predicting College Students' Placements Based on Academic Performance Using Machine Learning Approaches", International Journal of Modern Education and Computer Science (IJMECS), Vol. 15, No. 6, 2023.
- [6] J. N. Ndunagu, C. U. Ezeanya, B. O. Onuorah, J. C. Onyeakazi and E. Ukwandu, "A Chatbot Student Support System in Open and Distance Learning Institutions", Computers (MDPI), 14(3):96, 2025.
- [7] Angular Team, "Angular Documentation", <https://angular.dev>, 2026.
- [8] Ionic Team, "Ionic Framework Documentation", <https://ionicframework.com/docs>, 2026.
- [9] Ionic Team, "Capacitor — Cross-Platform Native Runtime", <https://capacitorjs.com/docs>, 2026.
- [10] Android Developers, "Build and Deploy Applications", <https://developer.android.com>, 2026.